Browser: Hi server 127.0.0.1:9999! Please give me the data at /api/posts and let me know if the client at **localhost:1111** can access it.

```
GET /api/posts HTTP/1.1
User-Agent: Chrome
Host: 127.0.0.1:9999
Accept: */*
Origin: http://localhost:1111
```

Server: Here is the data. And, yup, **any client** can have access.

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
```

This conversation represents the simplest dialogue that can take place between a browser and a server during a CORS request. As you'll see in subsequent chapters, this conversation grows richer and new headers are added based on the client's needs.

CORS is built around many headers, but the two most important are:

- The Origin request header
- The Access-Control-Allow-Origin response header

These headers must be present on every successful CORS request. Without one or the other, the CORS request will fail. Let's dig deeper into the vocabulary of these two headers.

3.4 Making a request with the Origin header

The Origin header is central to CORS. The client identifies itself to the server by using the Origin header. Think of it as the client's calling card.

A CORS request must have an Origin header. There is no way around that. If there is no Origin header it isn't CORS. With that in mind, let's revisit the sample app and take a look at the actual Origin header.

3.4.1 Viewing the Origin header

The browser's Network tab lets you view the HTTP headers included on the request, including the Origin header. View this header by opening the Network tab in the browser's JavaScript console and reloading the page at http://localhost:1111/client.html.

After the page finishes loading, choose the request to /api/posts in the Network tab. This is the actual CORS request from localhost:1111 to the API server on 127.0.0.1:9999. On the right are the HTTP headers for the request (if there are no headers be sure the Headers tab is selected). You should see the Origin header in the list of headers, as shown in figure 3.14.

Notice how the console only shows the request headers, and no response headers. Because the CORS request is failing, the browser hides the response information from the console. Once the server is configured to support CORS, the response information will appear here as well.

Also notice that while the Origin header is present on the request, the code in client.html never added it to the request. The Origin header is silently added to the request by the browser. Next let's take a look at what the Origin header is and how it appears in the request.